

CSC1003 Tutorial 4

Functions

Overview

- Brief Review
 - Defining functions
 - Calling functions
- Keyword “static”
 - Static methods
 - Static variables
- Keyword “public”
- Why using functions?

Review

1. Defining a function (basic structure):

- Function name:
 - snake_case
 - lowerCaseCamel
 - PascalCase
- Function type:
 - void (no return statement)
 - int
 - double
 - boolean
 - String
 - ...

```
1 public class Test { Function name
2     public static void HelloWorld() { Function type
3         System.out.println("Hello World");
4     }
5
6     Run | Debug
7     public static void main(String[] args) {
8         HelloWorld();
9     }
10 }
```

Review

1. Defining a function(basic structure):

- Arguments:
 - none or any
 - specified by (type name)
- Body:
 - one or several statements
- Return statement (necessary):
 - exit (for void functions)
 - exit with values (for non-void functions)

```
1 public class Test {
2     public static void HelloWorld(String message) {
3         System.out.println("Hello World " + message);
4     }
5
6     Run | Debug
7     public static void main(String[] args) {
8         HelloWorld("Hello CSC1003");
9     }
10 }
```

Arguments

Note that in Java, we could not define another function in a function body.

Review

2. Calling a function:

- Function name;
- Arguments.

```
1  public class Test {  
2      public static void HelloWorld(String message) {  
3          System.out.println("Hello World " + message);  
4      }  
5  
   Run | Debug  
6  public static void main(String[] args) {  
7      HelloWorld("Hello CSC1003");  
8  }  
9  }  
10
```

Function name Arguments

Keyword: static

1. Non-static methods could not be called in static methods directly.

Recall that the “main” function is static. Therefore, to call a non-static function in “main”, you need to **define an instance of the class** and call the function through the instance.

Eg. `Object.HelloWorld();`

```
1 public class Test {
2     // Could not be called directly.
3     void HelloWorld() {
4         System.out.println("Hello World");
5     }
6
7     // could be called directly
8     static void staticHelloWorld() {
9         System.out.println("Hello World");
10    }
11    Run | Debug
12    public static void main(String[] args) {
13        // Call by class
14        Test object = new Test();
15        object.HelloWorld();
16        object.staticHelloWorld();
17
18        // Call directly
19        HelloWorld(); // Error: non-static method cannot be referenced from a static context
20        staticHelloWorld();
21    }
22 }
```

Keyword: static

2. Non-static data could not be referenced in static methods.

While functions could be static or non-static, variables share the similar rules.

Calling: `Test.staticPrint();`

```
1 public class Test {
2     static int a = 1;
3     int b = 2;
4
5     void Print() {
6         System.out.println("a = " + a);
7         System.out.println("b = " + b);
8     }
9
10    static void staticPrint() {
11        System.out.println("a = " + a);
12        System.out.println("b = " + b); // Error: non-static variable count cannot be referenced from a static context
13
14        Test object = new Test();
15        System.out.println("b = " + object.b);
16    }
17    Run | Debug
18    public static void main(String[] args) {
19        Test object = new Test();
20        object.Print();
21
22        staticPrint();
23    }
24 }
```

Keyword: static

Whether the rules hold vice versa?

That is, can static methods/variables be referenced by non-static methods?

Try it by yourself!

“The static keyword is used to construct methods that will exist regardless of whether or not any instances of the class are generated. Any method that uses the static keyword is referred to as a static method.”

[Static Method in Java With Examples - GeeksforGeeks](#)

Keyword: public

1. Takes effect when there are multiple classes.
2. “public” v.s. “private”:
 - public functions: could be accessed by other classes;
 - private functions: could be accessed only within the class (by other functions).

```
class HelloWorld {  
    public static void publicStaticPrintHelloWorld() {  
        System.out.println("Hello world!");  
    }  
    private static void privateStaticPrintHelloWorld() {  
        System.out.println("Hello world!");  
    }  
}
```

HelloWorld.java

```
class Main {  
    public static void main(String[] args) {  
        HelloWorld.publicStaticPrintHelloWorld();  
        HelloWorld.privateStaticPrintHelloWorld(); // Error: method is not visible  
    }  
}
```

Main.java

Why using functions?

When designing programs, there are some design principles.

Encapsulation: encapsulate codes into reusable modules (classes) and functions, which reduces repetition and improves readability.

Example: determine prime numbers.

```
J Main.java > ...
1  class Main {
    Run | Debug
2      public static void main(String[] args) {
3          boolean isPrime = true;
4          for (int i = 2; i < 3; ++i) {
5              if (3 % i == 0) {
6                  isPrime = false;
7              }
8          }
9          System.out.println("3 is prime: " + isPrime);
10         isPrime = true;
11         for (int i = 2; i < 4; ++i) {
12             if (4 % i == 0) {
13                 isPrime = false;
14             }
15         }
16         System.out.println("4 is prime: " + isPrime);
17         isPrime = true;
18         for (int i = 2; i < 7; ++i) {
19             if (7 % i == 0) {
20                 isPrime = false;
21             }
22         }
23         System.out.println("7 is prime: " + isPrime);
24     }
25 }
26
```

```
1  class Main {
2      public static boolean isPrime(int n) {
3          for (int i = 2; i < n; ++i) {
4              if (n % i == 0) {
5                  return false;
6              }
7          }
8          return true;
9      }
    Run | Debug
10     public static void main(String[] args) {
11         System.out.println("3 is prime: " + isPrime(3));
12         System.out.println("4 is prime: " + isPrime(4));
13         System.out.println("7 is prime: " + isPrime(7));
14     }
15 }
16
```

Why using functions?

Good principals for designing a module:

- (1) Encapsulate codes into reusable functions;
- (2) Determine public and private functions;
- (3) Apply a consistent naming convention and write comments for your codes;
- (4) Write API documents for public functions which only expose how to call it without exposing the implementation details in it.

Practice more and form your own code style!

Practice: Swapping Elements

Let's write a method called `swapElements` that swaps two elements of an array. How can we do this?

What parameters should it take (if any)? What should it return (if anything)?

```
public void run() {  
    int[] array = new int[5];  
    ...  
    swapElements(array[0], array[1]);  
    ...  
}
```

```
private void swapElements(int x, int y) {  
    int temp = x;  
    x = y;  
    y = temp;  
}
```

Why it doesn't work?

Ints are primitives, so they are **passed by value!**

Their variable boxes store their actual values.

So changes to the parameter do NOT affect the original.

```
private void swapElements(int x, int y) {  
    int temp = x;  
    x = y;  
    y = temp;  
}
```

Another method

```
public void run() {  
    int[] array = new int[5];  
    ...  
    swapElements(array, 0, 1);  
    ...  
}
```

```
private void swapElements(int[] arr, int pos1, int pos2) {  
    int temp = arr[pos1];  
    arr[pos1] = arr[pos2];  
    arr[pos2] = temp;  
}
```

Why this method works?

Arrays are all **objects** in the java. And java passes all **objects** by reference.

When you pass an array to another method, actually the reference to that array is copied.

- Any changes in the content of the array through that reference will affect the original array.
- But changing the reference to point to a new array will not change the existing reference in the original method.

Q&A